Category C Problems

M1. Student Placement Record Management

Scenario

The T&P cell currently tracks student placements using three parallel arrays — names, companies, packages — and it has already gone out of sync twice this month. Rebuild it the OOP way.

Task

- Define a class PlacementRecord with fields String studentName, String company, and double packageLpa.
- Write a constructor that sets all three fields.
- Write an instance method printRecord() that prints one formatted line.
- Create three PlacementRecord objects for different students, store them in an array of PlacementRecord, and print each one in a loop.

Suggested method signature(s)

```
class PlacementRecord { ... }
void printRecord()
```

Sample Input / Output

Input	Output
Ravi, TCS, 4.5 *Anitha, Zoho, 6.2* *Karthik, Infosys, 4.0*	*Ravi -> TCS @ 4.5 LPA* *Anitha -> Zoho @ 6.2 LPA* *Karthik -> Infosys @ 4.0 LPA*

M2. Hostel Mess Wallet Management

Scenario

A hostel mess-card top-up wallet must never be allowed to go negative, and its balance must never be overwritten directly from outside the class.

Task

- Define a class MessWallet with a private double balance field.
- Write a public constructor that accepts an opening balance; if a negative value is given, start at 0 instead and print a warning.
- Write a public method topUp(double amount) that rejects amount <= 0 with a message, otherwise adds it to the balance.
- Write a public method deduct(double amount) that rejects any amount greater than the current balance, printing a clear message instead of allowing balance to go negative.
- Write a public getBalance() method for read-only access. There must be no public way to set balance directly from outside the class.

Suggested method signature(s)

```
void topUp(double amount)
void deduct(double amount)
double getBalance()
```

Sample Input / Output

Input	Output
opening = 500 *topUp(200)* *deduct(1000)*	*Balance after top-up: 700.0* *Deduct rejected: insufficient balance* *Final balance: 700.0*

M3. Course Credit Management

Scenario

Some courses come with a separate lab component and lab credit count; most don't. Support both without writing the same setup logic twice.

Task

- Define a class Course with fields code, title, credits, and labCredits.
- Write a constructor Course(String code, String title, int credits, int labCredits) that sets all four fields directly.
- Write a second constructor Course(String code, String title, int credits) for theory-only courses, which uses this(...) to chain to the four-argument constructor above with labCredits set to 0.
- Write a method totalCredits() returning credits + labCredits.
- Create one Course object with each constructor and print totalCredits() for both.

Suggested method signature(s)

```
public Course(String code, String title, int credits, int labCredits)
public Course(String code, String title, int credits)   // chains via this(...)
```

Sample Input / Output

Input	Output
21CSC201J, Data Structures, 4 (theory-only) *21CSC205L, DSA Lab, 3, labCredits 1*	*21CSC201J total credits: 4* *21CSC205L total credits: 4*

M4. Library ID Card Management

Scenario

Two “different” variables in a script both claim to represent Ravi's library card, but only one of them is actually real. Prove it with code.

Task

- Define a class IdCard with fields String name and int booksIssued, both set through a constructor.
- Create one IdCard object for Ravi. Assign a second variable to point at that same object (do not create a new one).
- Through the second variable, change booksIssued.
- Print the field's value as seen through the first variable, and print whether the two variables are == to each other.
- Create a third, separate IdCard object with identical field values, and print whether it is == to the first.

Suggested method signature(s)

```
class IdCard { String name; int booksIssued; }
```

Sample Input / Output

Input	Output
IdCard ravi = new IdCard("Ravi", 0); *IdCard duplicate = ravi;* *duplicate.booksIssued = 3;* *IdCard separate = new IdCard("Ravi", 3);*	*Ravi's booksIssued (via first variable): 3* *duplicate == ravi: true* *separate == ravi: false*

M5. Student and College Information Management

Scenario

A club member's first draft of Student stores attendance, name, AND the college name as instance fields — meaning every single student object ends up with its own copy of “SRM Institute of Science and Technology” typed in separately. Fix the design.

Task

- Define a class Student with instance fields name and attendance, and a static field collegeName shared by every student.
- Add a static field studentCount that increments once inside the constructor, every time.
- Write a static method printCollegeInfo() that prints collegeName and studentCount — and must not reference any instance field.
- Create two Student objects, then call printCollegeInfo() through the class name, not through either object.

Suggested method signature(s)

```
static void printCollegeInfo()
```

Sample Input / Output

Input	Output
2 Student objects created	*SRM Institute of Science and Technology* *Students created: 2*